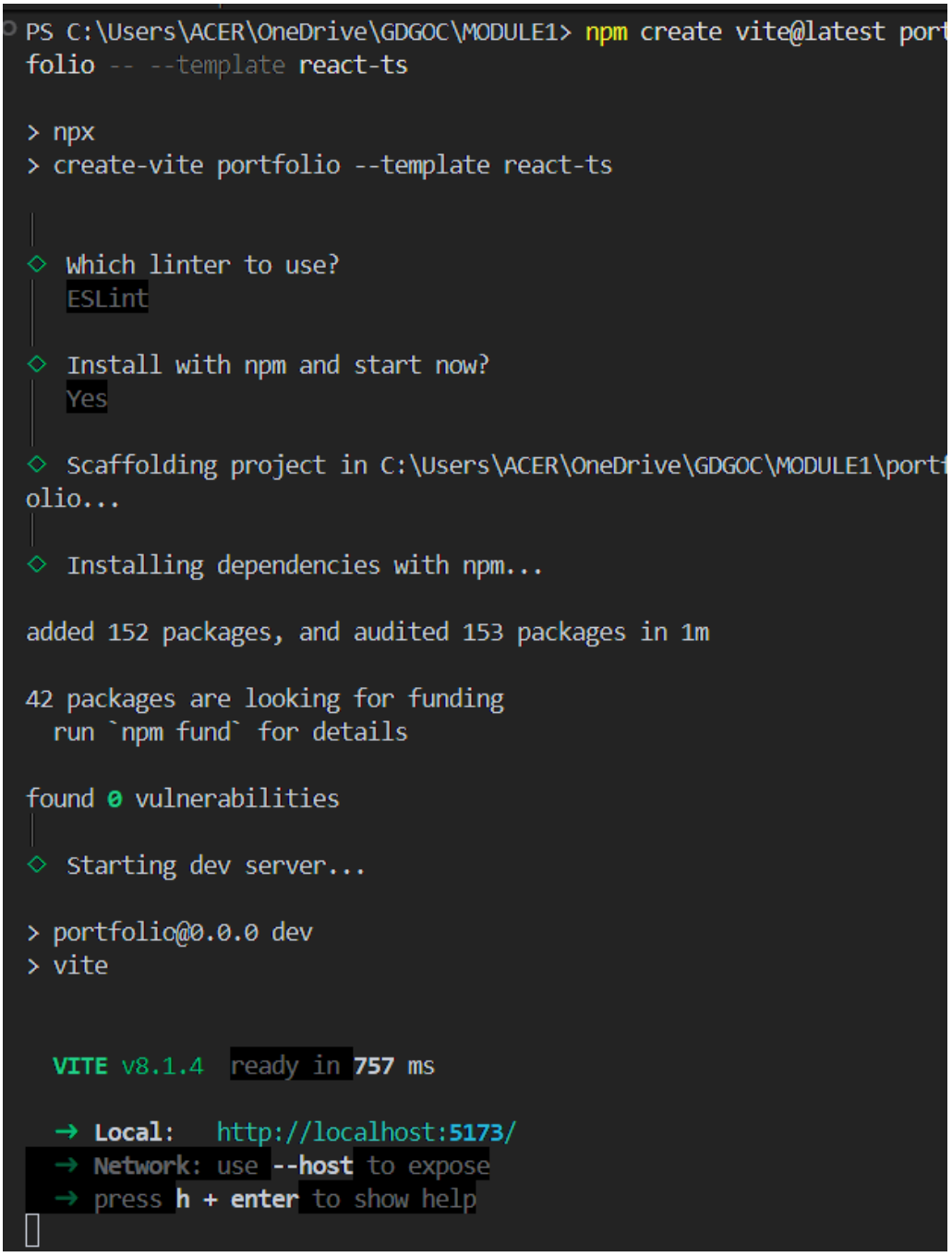
Notes Explore Module 2

Class	GDGOC
Date	July 17, 2026

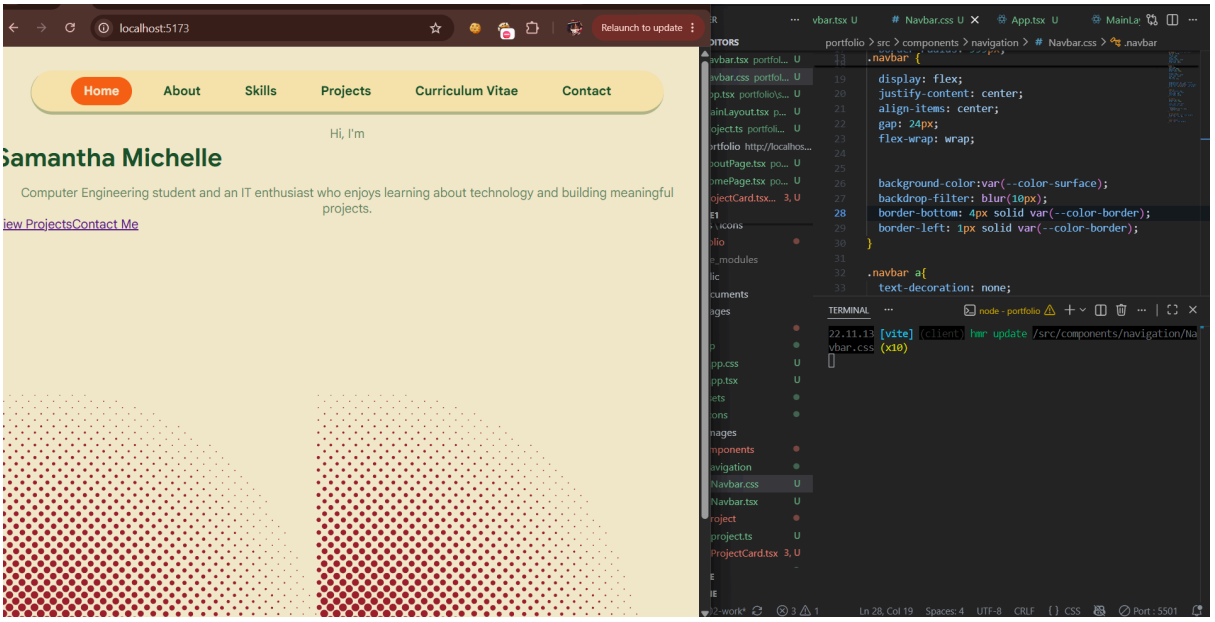
Progress Documentation

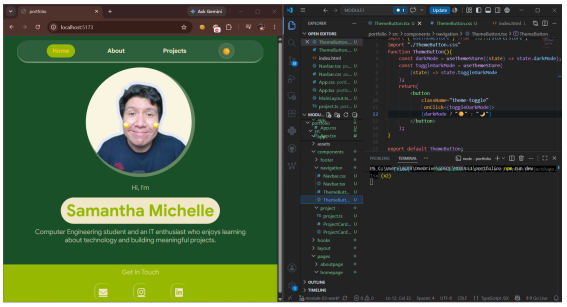
1. Setting up


PS C:\Users\ACER\OneDrive\GDGOC\MODULE1> npm create vite@latest portfolio -- --template react-ts
> npx
> create-vite portfolio --template react-ts
◇ Which linter to use?
ESLint
◇ Install with npm and start now?
Yes
◇ Scaffolding project in C:\Users\ACER\OneDrive\GDGOC\MODULE1\portfolio...
◇ Installing dependencies with npm...
added 152 packages, and audited 153 packages in 1m
42 packages are looking for funding
run `npm fund` for details
found 0 vulnerabilities
◇ Starting dev server...
> portfolio@0.0.0 dev
> vite
VITE v8.1.4 ready in 757 ms
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help

```
PS C:\Users\ACER\OneDrive\GDGOC\MODULE1\portfolio> npm install react-router-dom

up to date, audited 157 packages in 2s

43 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\ACER\OneDrive\GDGOC\MODULE1\portfolio> npm run dev

> portfolio@0.0.0 dev
> vite

13.45.40 [vite] (client) Re-optimizing dependencies because lockfile has changed

  VITE v8.1.4  ready in 508 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

2. Modificate module 01 project and design while implement routing





Summary

TypeScript = JavaScript + type rules.

JavaScript is very flexible because a variable can contain a number, string, object, or even null. In large projects, this flexibility can cause bugs that only appear when the website is running.

By using TypeScript, the data types are checked before the program runs, helping developers detect errors earlier. Its structure is similar to a `typedef struct`, where the expected properties and data types are defined clearly.

```
interface User {
  name: string;
  age: number;
}
```

```
function getUserAge(user: User): number {
  return user.age + 1;
}
```

By using TypeScript, errors can be detected before the program is executed. Without TypeScript, errors may only appear when the program is already running.

TypeScript code is compiled first and then automatically converted into JavaScript, because browsers cannot execute TypeScript directly.

In addition, using only JavaScript can result in silent failures. A silent failure does not necessarily cause the program to crash, but it may produce an incorrect result. For example, entering the wrong data type can produce NaN, such as when the program expects a number but receives a string instead.

TypeScript Concept

1. Type

A type defines the rules for what kind of data a value can contain.

```
let name: string = "Alice";
let age: number = 20;
```

```
let isStudent: boolean = true;
```

2. Interface

Defines the properties that an object has.

```
interface User {
  id: number;
  name: string;
  email: string;
}
```

3. Type Alias

Used to define a custom data type and restrict the allowed input values.

In this example, `status` can only be `active`, `inactive`, or `pending`. If the value is `"active"`, TypeScript will show an error.

```
type Status = "active" | "inactive" | "pending";
```

4. Generics

A generic function can work with many data types while still maintaining type safety.

For example:

```
const first = getFirst(["a", "b", "c"]);
```

TypeScript knows that the result will be a string or undefined if the array is empty.

If the array contains numbers, TypeScript will automatically infer the result as a number or undefined.

This means the input element type and the output type will always match.

Component-Driven Development (REACT)

1. Component

A small, reusable part of a user interface or website.

This is a simplified version of what was originally written like this:

```
<div class="project-card">
  <h3>Portfolio Website</h3>
  <p>A personal portfolio website built with HTML, CSS, and
JavaScript.</p>
  <a href="https://example.com">Live Demo</a>
</div>

<div class="project-card">
  <h3>Robot Controller</h3>
  <p>A robotics project using Arduino and motor controller.
</p>
  <a href="https://example.com">Live Demo</a>
</div>
```

To this

```
interface ProjectCardProps {
  title: string;
  description: string;
  techStack: string[];
  liveUrl?: string;
}

const ProjectCard: React.FC<ProjectCardProps> = ({
  title,
  description,
  liveUrl
}) => {
  return (
    <div className="project-card">
      <h3>{title}</h3>
      <p>{description}</p>
      {liveUrl && <a href={liveUrl}>Live Demo</a>}
    </div>
  );
```

```
};

<ProjectCard
  title="Portfolio Website"
  description="A personal portfolio website built with HTM
L, CSS, and JavaScript."
  liveUrl="https://example.com"
/>

<ProjectCard
  title="Robot Controller"
  description="A robotics project using Arduino and motor c
ontroller."
  liveUrl="https://example.com"
/>
```

2. Props

Props are data passed from outside into a component.

Based on the example above, the props are `title`, `description`, and `liveUrl`.

The question mark (`?`) means that the property is optional.

```
liveUrl && <a href={liveUrl}>Live Demo</a>
```

This means that if `liveUrl` exists, the link will be displayed. If it does not exist, nothing will be displayed.

```
ProjectCard: React.FC<ProjectCardProps>
```

This means that `ProjectCard` is a React Function Component, and its props must follow the structure defined in `ProjectCardProps`.

3. State

Data that can change within a component.

For example, the number of likes on a like button.

4. React Hooks

A special React function used to connect a component to React features.

Hook	Purpose
useState	Local component state
useEffect	Side effects (fetch data, subscriptions)
useCallback	Memoize functions
useMemo	Memoize computed values
useRef	Direct DOM access / mutable values
useContext	Access React Context

```
const [count, setCount] = useState(0);
```

`useState ←` The component now has a state variable called `count`. When the value of `count` changes, React re-renders the component.

```
function Button() {
  const theme = useContext(ThemeContext);
```

`useContext` → Gets data from a parent component without passing props through each component manually.

`useEffect` → Connects a component to systems outside React, such as APIs, `localStorage`, or the document.

`useMemo` → Caches the result of a calculation so it does not need to be recalculated unnecessarily.

`useCallback` → Caches a function so React can reuse the same function reference.

`useRef` → Accesses DOM elements or stores a value without causing the component to re-render when the value changes.

React Router

Cara untuk pindah ke page React yang berbeda

```
import { BrowserRouter, Routes, Route, Link } from "react-r
outer-dom";
```

`BrowserRouter` → The main wrapper that tells React the application uses routing.

`Routes` → A container for listing all available routes or pages.

`Route` → Defines which component should be displayed for a specific URL.

`Link` → A replacement for the `<a>` element that allows users to navigate between pages without reloading the website.

```
function App() {
return (
    <BrowserRouter>
        <nav>
            <Link to="/">Home</Link>
            <Link to="/projects">Projects</Link>
        </nav>
        <Routes>
            <Route path="/" element={<HomePage />} />
            <Route path="/projects" element={<ProjectsPage
/>} />
            <Route path="/projects/:id" element={<ProjectDe
tail />} />
        </Routes>
    </BrowserRouter>
    );
}
```

Zustand React

Zustand is a global state management library that allows state to be accessed from any component without repeatedly passing props and without creating a Provider—a wrapper that shares data with the components inside it.

Zustand can replace the combination of `useState` and `useContext` when managing shared state across multiple components.

For example, it can be used to create a dark and light mode toggle.

```
export const useThemStore = create<ThemeStore>((set)=> ({
    darkMode: false,
    toggleDarkMode: () =>
        set((state) => ({
            darkMode: !state.darkMode,
        })),

}));
```

`create<ThemeStore>` → Creates a store whose structure follows the `ThemeStore` interface.

`set` → A function provided by Zustand to update the state.

In the code above, `toggleDarkMode` uses `set` to update the state inside the store.